

Docker Learning Lab

Applicazione Flask + PostgreSQL + pgAdmin con Docker Compose

Obiettivo del laboratorio. Creare un piccolo ambiente applicativo realistico per capire il funzionamento di Docker Desktop, delle immagini, dei container, di Docker Compose, della rete tra servizi e dei volumi persistenti.

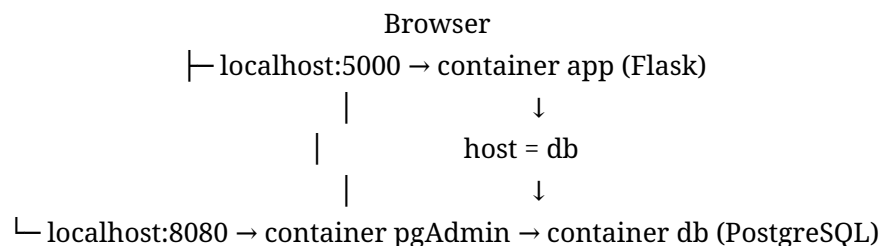
1. Idea del progetto

Il progetto è una piccola applicazione web in Python/Flask che salva nomi in un database PostgreSQL. L'utente può inserire un nome e verificare quante persone con lo stesso nome sono presenti nel database. I dati non vengono mostrati direttamente nella pagina, ma restano consultabili tramite pgAdmin.

Servizio	Ruolo	Accesso
app	Applicazione Flask	http://localhost:5000
db	Database PostgreSQL	Rete interna Docker, porta 5432
pgadmin	Interfaccia grafica PostgreSQL	http://localhost:8080

2. Architettura Docker

Docker Compose descrive l'intero ambiente. Ogni componente gira in un container separato, ma i container appartengono alla stessa rete Docker e possono comunicare usando il nome del servizio.



3. Ruolo dei file principali

- **app/app.py:** Contiene la logica Flask e le query SQL verso PostgreSQL.
- **requirements.txt:** Elenca le dipendenze Python, in particolare Flask e psycopg2-binary.
- **Dockerfile:** Descrive come costruire l'immagine personalizzata dell'applicazione.
- **docker-compose.yml:** Descrive i tre servizi, le porte, le variabili d'ambiente, la rete e i volumi.
- **.env:** Contiene le credenziali locali e le variabili che non devono essere pubblicate su GitHub.
- **.gitignore:** Evita che file locali e sensibili, come .env, vengano versionati.
- **README.md:** Documenta il progetto nel repository GitHub.

4. Dockerfile: cosa abbiamo fatto

```
FROM python:3.13-slim
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY app/ .
```

```
CMD ["python", "app.py"]
```

Il Dockerfile parte da un'immagine Python ufficiale, crea una directory di lavoro nel container, installa le dipendenze e copia il codice dell'app. Il comando CMD specifica cosa deve essere eseguito quando il container viene avviato.

5. Docker Compose: orchestrazione dell'ambiente

Docker Compose permette di avviare più container con un unico comando. Nel laboratorio abbiamo definito i servizi app, db e pgadmin. La variabile DB_HOST usa il valore 'db': non si usa localhost perché Flask deve raggiungere un altro container, non se stesso.

Il comando principale usato è:

docker compose up --build

L'opzione --build ricostruisce l'immagine dell'applicazione quando il codice o il Dockerfile cambiano. Il comando 'docker compose down' ferma e rimuove i container dello stack, ma non elimina i volumi.

6. Persistenza dei dati con i volumi

I container sono pensati per poter essere ricreati. Per questo i dati importanti non devono dipendere dal filesystem interno del container. Abbiamo usato due volumi:

- **postgres_data:** mantiene i dati reali di PostgreSQL anche se il container db viene ricreato.
- **pgadmin_data:** mantiene la configurazione di pgAdmin e i server registrati.

Attenzione: docker compose down -v elimina anche i volumi e quindi può cancellare i dati persistenti del laboratorio.

7. Docker Desktop: cosa abbiamo osservato

Docker Desktop è stato usato come interfaccia grafica per osservare l'ambiente Docker. La schermata Containers permette di vedere lo stack app_docker_learning e i singoli container. Per ogni container sono visibili immagine, porte pubblicate, stato, utilizzo CPU e azioni di start/stop.

☐	app_docker_learning	-	-	-	0.6%	7 minutes ago				
☐	db-1	68a82480238a	postgres:16		0%	7 minutes ago				
☐	pgadmin-1	21b8523d1ce2	dpage/pgadmin4	8080:80	0.58%	7 minutes ago				
☐	app-1	de5a627fa3e8	app_docker_learning-app	5000:5000	0.02%	7 minutes ago				

Showing 5 items

Figura 1 - I tre container del laboratorio visualizzati in Docker Desktop.

Nello screenshot si distinguono: db-1 basato su postgres:16, pgadmin-1 basato su dpage/pgadmin4 e app-1 basato sull'immagine personalizzata app_docker_learning-app. Le porte 8080:80 e 5000:5000 mostrano il mapping tra il PC Windows e i servizi nei container.

8. pgAdmin dentro Docker

pgAdmin non è stato installato direttamente su Windows: anche lui gira in un container. Dal browser si accede a pgAdmin su localhost:8080. Per collegarsi a PostgreSQL si usa come host 'db', perché entrambi i servizi sono nella rete creata da Docker Compose.

Da pgAdmin è possibile esplorare Databases → dockerdb → Schemas → public → Tables → persone, oppure usare il Query Tool con query come `SELECT * FROM persone`.

9. Gestione delle credenziali

Inizialmente le credenziali demo erano scritte direttamente nel docker-compose.yml. Per seguire una pratica più corretta, sono state spostate in un file .env escluso da Git tramite .gitignore. Il compose usa quindi riferimenti come `${POSTGRES_PASSWORD}`. Questo evita di pubblicare credenziali reali nel repository.

10. Collegamento con GitHub

Il progetto è stato inizializzato come repository Git e pubblicato su GitHub come esempio pubblico di un'applicazione multi-container. Il repository documenta l'uso di Dockerfile, Docker Compose, networking, volumi persistenti, PostgreSQL e pgAdmin.

```
git init
```

```
git add .
```

```
git commit -m "Initial Docker learning project"
```

```
git branch -M main
```

```
git push -u origin main
```

11. Concetti Docker appresi

- Differenza tra immagine e container.

- Pull di immagini da Docker Hub.
- Build di un'immagine personalizzata tramite Dockerfile.
- Esecuzione di più servizi con Docker Compose.
- Comunicazione tra container tramite rete Docker e nome del servizio.
- Port mapping tra host Windows e container.
- Persistenza dei dati mediante volumi.
- Separazione tra codice applicativo, database e strumenti di amministrazione.
- Ricostruzione dell'immagine dopo modifiche al codice.
- Uso di variabili d'ambiente e file .env.
- Osservazione e gestione visuale dei container tramite Docker Desktop.

12. Perché questo laboratorio è utile per Coccola

Il laboratorio replica in scala ridotta il tipo di architettura che potrà essere usato per Coccola. L'applicazione potrà essere trasformata in un'immagine Docker, mentre PostgreSQL resterà un servizio separato con dati persistenti. In futuro l'immagine applicativa potrà essere pubblicata in un registry privato e scaricata sulla VPS, mantenendo separati codice, configurazione e dati.

In sintesi, il laboratorio non serve solo a 'far partire Docker': serve a comprendere il modello operativo con cui un'applicazione può essere costruita, versionata, ricreata e successivamente distribuita in modo coerente.